

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División Ciencias de la Ingeniería
Sistemas Operativos 2



“Proyecto Final”

Kenny Marco Augusto López Salazar **202031554**

Quetzaltenango 27 de Octubre 2026

1. Alcance y revision del enunciado

Se reviso el PDF Proyecto Final(1).pdf, sus nueve paginas, y el codigo de la version 0.7. La solucion cubre recursos limitados, clientes, niveles de servicio, pedidos manuales y continuos, cinco tipos de mercancia, politicas comparables, recepcion e inventario independientes, ubicaciones no contiguas, interbloqueo deliberado, interfaz y backend Java con sincronizacion real. Los entregables obligatorios son codigo fuente, manual de usuario y manual tecnico. PostgreSQL y Docker son decisiones adicionales del proyecto. No se exige autentificacion por contrasena en el enunciado.

La cobertura funcional no significa garantizar plazos ante stock inexistente o sobrecarga: el sistema mide y muestra incumplimientos. Para generar los cinco niveles deben existir cuentas de esos niveles. El limite de 100 corresponde al experimento comparativo y al historial visible, no al total de pedidos de la operacion continua.

2. Arquitectura por componentes

React consulta GET /api/state cada 700 ms y envia comandos POST. Nginx sirve el bundle estatico. Main.java valida parametros y traduce errores a HTTP. Simulation.java contiene el orquestador, pedidos, cuentas, recursos y areas. Warehouse.java encapsula celdas, lotes, reservas y consumo. Scheduling.java comparte reglas de orden y SLA con Comparison.java. StateStore.java abstraee persistencia y PostgresStore usa JDBC. Json.java produce las respuestas JSON. El frontend presenta snapshots; no ejecuta la logica de concurrencia ni calcula las asignaciones de recursos.

No se utiliza Spring Boot, JPA ni una arquitectura de microservicios. Se eligio el servidor HTTP del JDK para concentrar la defensa en hilos, semaforos, mensajes y almacenamiento. Recepcion e Inventario son areas independientes implementadas como hilos del mismo proceso Java; su mensajeria no es IPC entre procesos nativos.

3. Infraestructura reproducible

Compose inicia PostgreSQL 18-alpine, backend compilado con JDK 21 y frontend compilado con Node y servido por Nginx. El healthcheck pg_isready condiciona el arranque del backend. Internamente JDBC se conecta a db:5432/redxela; externamente PostgreSQL se expone en 5439, la API en 8090 y el frontend en 5173. El volumen redxela_pgdata se monta en /var/lib/postgresql. Las credenciales locales predeterminadas son usuario redxela y clave redxela_local; .env permite configurar DATABASE_PASSWORD antes de crear la base. El controlador incluido es pgJDBC 42.7.14.

Mantener el mismo proyecto Compose conserva el volumen. Un cambio de carpeta puede cambiar el nombre del proyecto y aparentar perdida de datos. down -v borra el volumen. Cambiar .env no cambia automaticamente la clave de una base ya inicializada. VITE_API_URL fija la direccion de la API al compilar; localhost corresponde a la misma PC, no a un despliegue remoto.

4. Modelo de clientes y pedidos

Client tiene ID, nombre, correo unico normalizado y nivel actual. Order conserva una copia del cliente y el servicio al crearse, por lo que un cambio posterior del cliente no reescribe su historia. El pedido almacena ID secuencial, lineas producto/unidades, total, etapas, estado, motivo, reserva, tiempos y escala SLA fija. Las lineas repetidas se consolidan y se valida que el total no supere las 600 unidades del almacen. Los pedidos antiguos sin cliente se identifican como Pedido de version anterior. El generador funciona cada segundo, elige una cuenta registrada y crea una linea de un producto del catalogo con 1 a 12 unidades. No impone un tope fijo de pedidos; la memoria y el checkpoint son limites practicos.

5. Catalogo, mercancias y etapas

El catalogo tiene diez productos: Cuaderno GENERAL, Vaso de vidrio FRAGIL, Saco de cemento PESADO, Silla VOLUMINOSO, Audifonos ELECTRONICO, Camiseta GENERAL, Espejo FRAGIL, Motor industrial PESADO, Mesa VOLUMINOSO y Teclado ELECTRONICO. Cada tipo tiene descripcion y necesidades diferentes. GENERAL pasa por empaque; FRAGIL agrega calidad; PESADO necesita montacargas y no empaque; VOLUMINOSO combina montacargas y empaque; ELECTRONICO requiere calidad sin empaque. Todos requieren bodega y despacho. En pedidos mixtos se toma la union de necesidades.

Preparacion: 150 ms/unidad, bodega y montacargas si aplica. Empaque: 250 ms/unidad, bodega y empaque cuando aplica. Calidad: 180 ms/unidad, bodega y calidad cuando aplica. Escaneo de despacho: 40 ms/unidad, bodega y escaner. Carga: 80 ms/unidad, bodega y carga, mas montacargas si aplica. Son duraciones de simulacion, no limites SLA. Mayor cantidad significa mayor trabajo. Escaneo y carga estan separados para no mantener el escaner durante toda la carga. Dos pedidos generales pueden competir por bodega y empaque; pesados y voluminosos por bodega y montacargas.

6. Hilos reales y exclusion mutua

Un ScheduledExecutorService ejecuta el planificador cada 50 ms y el generador cada segundo. Un pool de 16 trabajadores ejecuta etapas. Un pool de dos hilos ejecuta Recepcion e Inventario. El servidor HTTP tiene ocho trabajadores. La demostracion de conflicto agrega dos hilos dedicados. synchronized(gate) protege cambios del dominio, asignaciones, snapshots y checkpoints para que una lectura no observe reservas o permisos parcialmente actualizados.

Los trabajadores duermen fuera del monitor mientras conservan permisos de sus recursos. Esto permite que otros pedidos y areas progresen. La escritura JDBC ocurre bajo el monitor: facilita atomicidad, pero puede reducir rendimiento. Cada etapa ordinaria requiere un encargado de bodega y hay diez, por lo que la capacidad limita etapas simultaneas. La cola del executor no sustituye la lista de espera del dominio.

7. Semaforos y prevencion de interbloqueo

Los grupos operativos tienen capacidades exactas: BODEGA 10, MONTACARGAS 3, EMPAQUE 6, CALIDAD 4, ESCANER 1 y CARGA 2; suman 26 unidades. Cada grupo usa Semaphore y arreglos de propietarios y tiempos de ocupacion por unidad. CONTROL_CONFLICTO tiene un permiso auxiliar y se usa exclusivamente en la demostracion; no modifica las capacidades exigidas.

La operacion normal comprueba disponibilidad de todos los recursos de una etapa bajo el mismo monitor y solo entonces adquiere el conjunto sin espera bloqueante. Si no puede, el pedido conserva su lugar de espera sin retener parte de las estaciones. Esta eliminacion de retencion parcial rompe hold-and-wait. finally libera permisos y propietarios incluso ante interrupcion o error. La politica de admision la controla el planificador; tryAcquire sin espera no garantiza FIFO aunque el semaforo sea fair=true. La prioridad no expropia trabajo iniciado.

8. Planificacion y envejecimiento

Scheduling.rank define PRIORIDAD por nivel, FIFO por ID, UNIDADES por total y ENVEJECIMIENTO por prioridad efectiva. El desempate usa ID. Se considera disponibilidad de stock y recursos, de modo que otro pedido apto puede avanzar si el primero no puede hacerlo. La comparacion captura un mismo instante de evaluacion para conservar un orden estable.

Prioridad efectiva = $\max(1, \text{nivel original} - \text{piso}(\text{espera acumulada en ms} / (\text{intervalo en segundos} * 1000)))$. El intervalo inicial es 30 segundos y admite 1 a 3600. Se suma espera entre etapas, por stock, recursos y conflicto; el procesamiento no produce ascensos. El servicio original no cambia. Envejecimiento reduce riesgo de inanicion por prioridad estricta, pero no garantiza stock ni capacidad bajo demanda ilimitada. Al llegar todos a nivel 1 se aplica llegada.

9. Mensajes entre Recepcion e Inventario

BlockingQueue de capacidad 8 transporta ReceiptEvent inmutable con identificador, producto y unidades. Recepcion escanea con el mismo ESCANER que despacho y envia el evento; Inventario procesa a su propio ritmo y es quien modifica Warehouse. El estado comun de recepciones pertenece al orquestador. No se comparten estructuras privadas entre las dos areas.

El POST confirma una recepcion durable antes de responder 202. Recepcion tarda $100 + 5 \cdot \text{unidades}$ ms y el consumidor 1200 ms por evento, permitiendo producir mas rapido que consumir. offer no bloquea indefinidamente cuando el canal esta lleno: se conserva el pendiente durable y se reintenta. El escaner se libera antes de esperar canal o espacio. lotId evita registrar dos veces una recepcion repetida. Una cola acotada sola perderia pendientes al reiniciar; por eso existe el respaldo durable.

10. Almacen y reservas atomicas

Warehouse define 60 celdas de 10 unidades: 600 en total. Es capacidad abstracta por unidad, no metros cubicos. Cada lote se relaciona con varias asignaciones celda/producto/cantidad/reserva. El recorrido ($\text{posicion} * 17$) modulo 60 visita todas las celdas porque 17 y 60 son coprimos y permite distribuir lotes no contiguos. Es una tabla de ubicaciones similar a asignacion indexada de bloques.

La recepcion cabe completa o queda esperando espacio sin modificar parcialmente las celdas. Antes de reservar se comprueban todas las lineas del pedido: si falta una, no se reserva ninguna. Si hay disponibilidad se reserva FIFO entre asignaciones. La reserva persiste entre etapas y el consumo ocurre solo al completar despacho. Invariantes: $0 \leq \text{reservado} \leq \text{fisico}$, $\text{disponible} = \text{fisico} - \text{reservado}$, $\text{usado por celda} \leq 10$ y $\text{total usado} \leq 600$. Reservar no equivale a retirar fisicamente.

11. Interbloqueo deliberado y recuperacion

Se crean dos pedidos con stock reservado. Un hilo adquiere ESCANER y espera CONTROL_CONFLICTO; el otro adquiere CONTROL_CONFLICTO y espera ESCANER. CountdownLatch coordina que ambos retengan el primer permiso antes de intentar el segundo. Se cumplen exclusion mutua, retencion y espera, ausencia de expropiacion espontanea y espera circular. La interfaz muestra propietarios, ciclo, motivo y eventos reales.

La decision manual interrumpe cooperativamente al perdedor. Su finally libera el primer permiso y lo envia a CONFLICTO_REINTENTO. El ganador puede adquirir ambos, terminar la demostracion y liberarlos. Ambos vuelven al flujo normal manteniendo sus reservas; no se usa Thread.stop ni se descuenta stock en la demostracion. El ganador resuelve primero el ciclo, sin promesa de finalizar antes todas las etapas. Reiniciar durante conflicto reconstruye recursos libres y recupera pedidos en la cola ordinaria, sin restaurar el deadlock.

12. Tiempo, SLA y estadisticas

System.nanoTime mide intervalos monotonos; no se usa la hora del calendario para duraciones. Cada pedido acumula espera y procesamiento separados. Los promedios generales usan completados y tiempos reales. El progreso usa trabajo estimado realizado y el restante excluye colas futuras. Ocupacion = $\text{tiempo ocupado acumulado de todas las unidades} / (\text{capacidad} * \text{tiempo observado})$, incluyendo asignaciones activas. La ocupacion reinicia al reiniciar el proceso.

Espera virtual en ms = $\text{espera real acumulada en ms} * \text{escala capturada por el pedido}$. La escala inicial es 60 y admite 1 a 3600. El cambio afecta pedidos nuevos y no acelera los trabajadores ni el envejecimiento. Limites: 1, 10, 30, 60 y 120 minutos. El primer valor es una interpretacion explicita de casi instantaneo, porque el enunciado no da una cifra. La igualdad exacta cumple y solo superar incumple. Pendientes: EN_PLAZO/VENCIDO; completados: CUMPLIDO/INCUMPLIDO. Historial anterior: SIN_MEDICION, excluido de cumplimiento. Se mide

espera acumulada, no tiempo total de respuesta; no se garantiza cumplimiento en sobrecarga.

13. Comparacion reproducible

Comparison usa eventos discretos en memoria. No inicia los trabajadores reales para medir cada politica. Comparte Scheduling, las etapas y las seis capacidades operativas con Simulation. La misma semilla genera la misma carga: 10 a 100 pedidos, niveles repartidos entre 1 y 5, producto aleatorio, 1 a 6 unidades y llegadas cada 100 ms. Se supone stock suficiente al inicio; no hay recepciones ni control auxiliar en el experimento.

Cada evento libera etapas terminadas, evalua pedidos que llegaron, ordena por politica y admite conjuntos completos de recursos. El reloj avanza a la siguiente llegada o finalizacion. Los resultados incluyen esperas, duracion, incumplimientos por servicio, trazas de finalizacion y ocupacion. Las cuatro politicas reciben la misma carga, escala e intervalo. Se guarda solo el ultimo informe sin modificar la politica, inventario o pedidos operativos. Esto permite comparacion controlada; no representa latencia real de JVM, PostgreSQL, sistema operativo ni falta de stock. Ninguna politica se declara universalmente mejor.

14. Persistencia PostgreSQL

StateStore tiene implementacion de memoria para pruebas y PostgreSQL como modo predeterminado. PostgresStore mantiene un bloqueo consultivo de sesion para evitar dos motores sobre la misma corrida. V1__schema.sql se aplica de forma idempotente; no usa Flyway. simulation_checkpoint tiene una fila autoritativa con formato 3, payload BYTEA, projection JSONB y fecha de actualizacion. El payload serializa dominio, reservas, almacen, cuentas, politica, IDs, envejecimiento, escala y ultimo experimento.

BYTEA y JSONB se confirman en una misma transaccion UPSERT mediante PreparedStatement. Las vistas inventory_products, warehouse_locations, warehouse_lots, receipt_events, distribution_orders y customer_accounts permiten consulta SQL. No son tablas JPA ni una base normalizada por entidad. La proyeccion es de consulta: editarla no cambia correctamente el dominio. Ante error se hace rollback y se pausa el motor; no se confirma una creacion si falla su persistencia. Se elige checkpoint completo por sencillez y atomicidad; su costo crece con toda la historia.

15. Recuperacion, cierre y excepciones

Al restaurar, completados siguen completados y pendientes conservan sus reservas; una etapa interrumpida se repite desde el inicio. Hilos, semaforos y propietarios son transitorios y se reconstruyen. Recepciones escaneadas pendientes se reencolan; lotId protege idempotencia. El generador reinicia detenido. La espera pendiente se incorpora una vez y no cuenta el tiempo apagado. Un checkpoint periodico de espera puede perder hasta el tramo desde la ultima confirmacion, aproximadamente cinco segundos, ante cierre abrupto.

El cierre desactiva nuevas asignaciones, interrumpe y espera hilos del conflicto y trabajadores y guarda estado. InterruptedException restituye la marca de interrupcion; finally devuelve permisos. RejectedExecutionException devuelve recursos si el pool rechaza trabajo. Parametros invalidos reciben 400; ruta inexistente 404; metodo incorrecto 405; errores inesperados 500 con registro en consola. Si falla PostgreSQL, se requiere restablecerlo y reiniciar desde el ultimo checkpoint confirmado. No se intenta continuar una corrida con escrituras ambiguas.

16. API y configuracion

GET /api/state retorna snapshot consistente de cuentas, productos, tipos, recursos, pedidos, recepciones, almacen y metricas. POST /api/clients usa name, email, level; /api/clients/service usa clientId, level; /api/orders usa clientId, items; /api/receipts usa productId, units; /api/policy usa value; /api/generator usa enabled; /api/aging usa seconds; /api/clock usa rate; /api/comparison usa seed, count; /api/conflict/start usa clientId, productId, units; /api/conflict/resolve usa winner. Los parametros van en query, no en un cuerpo JSON.

Variables: DATABASE_URL, DATABASE_USER, DATABASE_PASSWORD, PORT y SCHEMA_PATH. REDXELA_MODE=memory activa pruebas sin persistencia; no es respaldo automatico si falla PostgreSQL. SIMULATION_TIME_SCALE acelera trabajo para pruebas, separado de la escala SLA de la interfaz. CORS abierto es una decision de desarrollo local y no una configuracion de produccion autenticada.

17. Validacion y defensa

SimulationTest valida recursos y politicas; InventoryTest reservas, capacidad, canal lleno, recuperacion y fallos; DeadlockTest ambos ganadores, rechazo y liberacion; ClientTest cuentas y servicio historico; AgingTest ascensos, desempate y conservacion; SlaTest limites y escala fija; ComparisonTest reproducibilidad, balance temporal, ocupacion y ausencia de cambios operativos; PostgresIntegrationTest persistencia JDBC. La version previa paso la suite y el usuario reporto pruebas locales satisfactorias. Las capturas verifican ejemplos visibles, no todas las invariantes ni una prueba de carga ilimitada. La integracion de desarrollo uso PGlite; la validacion Docker/PostgreSQL nativo corresponde al entorno local del usuario.

Defensa sugerida: explicar primero reserva atomica y admision sin retencion parcial; despues el ciclo deliberado y su ruptura; luego mensajeria durable y lotes no contiguos; por ultimo distinguir envejecimiento real, SLA virtual y comparacion calculada. Mostrar cinco cuentas para generador, cola por stock, recepcion y reactivacion, propietarios, conflicto y recuperacion, y experimento repetido.

18. Limites y decisiones justificadas

La simulacion mantiene toda la corrida en memoria y reescribe el checkpoint completo; no esta pensada como logistica de produccion sin limites fisicos. El historial visible y el experimento son acotados, aunque la generacion operativa no tenga un maximo fijo. El control auxiliar hace el deadlock facil de reproducir sin alterar capacidades requeridas. Las cuentas carecen de autenticacion porque el enunciado pide registro y servicio, no login. La comunicacion entre areas es interhilo, coherente con backend Java basado en hilos, no procesos nativos separados.

Si la evaluacion interpreta comunicacion entre procesos como obligacion de procesos del sistema operativo distintos, o exige medir las politicas con hilos en vez de un experimento calculado, se necesita confirmar ese criterio con el catedratico: esos alcances adicionales no estan explicitados en las secciones operativas. El manual declara la implementacion real para evitar atribuirle mecanismos inexistentes. El codigo y los resultados visibles cubren el flujo solicitado con estas decisiones de diseno.

19. Comandos de operacion y pruebas

```
docker compose up --build
docker compose ps
docker compose logs backend
docker compose down
docker compose exec db psql -U redxela -d redxela -c "SELECT * FROM inventory_products;"
```

Pruebas sin BD externa: desde backend, con JDK 21 y bash. Para PowerShell compile con javac -d out (Get-ChildItem src/*.java,tests/*.java).FullName. El classpath de Windows usa punto y coma.

```
javac -d out src/*.java tests/*.java
java -cp out SimulationTest
java -cp out InventoryTest
java -cp out DeadlockTest
java -cp out ClientTest
java -cp out AgingTest
java -cp out SlaTest
java -cp out ComparisonTest
```

PostgresIntegrationTest debe usar una base nueva y vacia, nunca la base redxela con datos de la demostracion. TEST_DATABASE_URL selecciona esa base; incluya lib/* en el classpath para JDBC.

```
# Ejemplo bash: base vacia creada previamente
export TEST_DATABASE_URL=jdbc:postgresql://localhost:5439/redxela_test
java -cp "out:lib/*" PostgresIntegrationTest
```